

Relatório Técnico: Infraestrutura de Microsserviços com Tolerância a Falhas e Monitoramento em Tempo Real

Wanderson de O. Silva¹, Yuri M. da Silva¹, Mateus A. Batista¹, Breno da P. Leal¹

¹Universidade Federal do Píauí (UFPI)

{Wanderson, Yuri, Mateus, Breno}@gmail.com

1. Introdução

A evolução das arquiteturas de software para modelos distribuídos trouxe a necessidade de garantir alta disponibilidade, tornando a aplicação de técnicas de tolerância a falhas um requisito crítico. Nesse contexto, a orquestração de contêineres com ferramentas como o Kubernetes destaca-se ao fornecer mecanismos nativos para manter a estabilidade das aplicações, como a auto-recuperação (auto-healing), que assegura a recriação automática de pods em caso de interrupções ou falhas. Além de lidar com instabilidades, a infraestrutura deve ser capaz de se adaptar de forma elástica à demanda computacional por meio do escalonamento horizontal, utilizando controladores como o Horizontal Pod Autoscaler (HPA) para ajustar dinamicamente o número de instâncias com base no uso de CPU.

Para que a automação dessas ações de resiliência seja confiável, é imprescindível estabelecer uma camada robusta de observabilidade. Ferramentas como o Prometheus são fundamentais para realizar o monitoramento remoto de clusters, coletando métricas do sistema em tempo real. Através desse monitoramento contínuo, é possível auditar dados cruciais para a operação, como o número de pods ativos, as oscilações no uso de CPU, as ações disparadas pelo HPA e o estado atual de cada instância (como Running, Failed ou Pending), garantindo a visibilidade necessária para validar o comportamento do ecossistema sob condições normais e de estresse.

2. Proposta e Arquitetura

A proposta deste trabalho consiste na modelagem e implantação de uma infraestrutura distribuída em ambiente local, projetada para suportar alta disponibilidade e escalabilidade dinâmica. Para assegurar o isolamento das responsabilidades e simular uma arquitetura de produção real, a topologia da rede foi dividida em dois nós físicos distintos, separando o plano de execução da aplicação da camada de observabilidade e monitoramento remoto. O sistema divide-se em dois nós físicos interligados:

- **Plano de Execução (Notebook A):** Hospeda o cluster Kubernetes (Minikube), sendo responsável pela orquestração da aplicação e pelo gerenciamento autônomo do ciclo de vida dos recursos.
- **Plano de Observabilidade (Notebook B):** Atua como servidor de monitoramento remoto via Prometheus, coletando dados de telemetria sem interferir na carga de trabalho do cluster.

A arquitetura foca em três pilares: a *auto-recuperação*, que delega ao Kubernetes a detecção e correção de falhas em instâncias; a *elasticidade*, através do escalonamento horizontal baseado em demanda; e a *visibilidade*, garantida pela coleta contínua de

métricas de estado e performance. Para validar esses pilares, o sistema foi desenhado para suportar testes de injeção de falhas (deleção de pods) e estresse computacional (picos de CPU).

3. Implementação e Desenvolvimento

A implementação do sistema foi dividida em duas frentes operacionais distintas, conectadas via rede local para permitir a observabilidade remota. A seguir, detalham-se as configurações de cada ambiente.

3.1. Configuração do Ambiente de Execução (Notebook A)

No Notebook A, foi estabelecido um cluster Kubernetes via Minikube para hospedar a aplicação e os exportadores de métricas.

- **Aplicação Principal (web-app):** Foi implantado um servidor PHP-Apache configurado inicialmente com duas réplicas para garantir a disponibilidade básica. A aplicação possui limites de recursos definidos (solicitação de 100m de CPU e limite de 200m) para permitir o cálculo preciso de escalonamento. O acesso externo é realizado através de um serviço do tipo `NodePort`, exposto na porta 30007.
- **Escalonamento Horizontal (HPA):** O controlador `web-app-hpa` foi configurado para monitorar o Deployment da aplicação. O gatilho de escalonamento foi definido em 50% da utilização média de CPU, permitindo que o cluster oscile dinamicamente entre o mínimo de 2 e o máximo de 6 réplicas.
- **Exposição de Métricas do Cluster:** Para fornecer dados detalhados ao Prometheus, utilizou-se o `kube-state-metrics`. Este componente foi implantado no namespace `kube-system` com permissões de leitura em nível de cluster (`ClusterRole`) para listar e monitorar recursos como pods, nós e HPAs.

3.2. Configuração do Monitoramento Remoto (Notebook B)

O Notebook B hospeda o servidor Prometheus, que atua como o coletor central de métricas. A configuração de coleta (*scrape*) foi parametrizada para buscar dados no IP do Notebook A.

- **Intervalo de Coleta:** Definido em 5 segundos para permitir uma visualização quase em tempo real das alterações no cluster.
- **Alvos de Coleta (Scrape Targets):**
 - **kube-state-metrics:** Coleta o estado dos objetos do Kubernetes (pods ativos, estados de réplicas).
 - **cAdvisor:** Obtém métricas de uso de recursos (CPU e Memória) diretamente dos contêineres via proxy do API Server.
 - **API Server:** Monitora a saúde e a performance do plano de controle do cluster.

4. Execução e Resultados

Nesta seção, são apresentados os resultados obtidos durante a implantação e validação da infraestrutura de microsserviços. O objetivo é demonstrar, de forma prática, a eficácia dos mecanismos de tolerância a falhas, do escalonamento horizontal e do monitoramento remoto. Para isso, os procedimentos foram divididos de acordo com a topologia da rede, separando o ambiente de execução da aplicação da camada de observabilidade.

4.1. Inicialização do Cluster e Implantação da Aplicação (Notebook 01)

O processo foi iniciado com a criação do cluster Kubernetes via Minikube e a habilitação do servidor de métricas, requisito obrigatório para o funcionamento do escalonamento baseado em CPU, como mostrado na figura 1.

```
PS C:\WINDOWS\system32> minikube start
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Automatically selected the docker driver
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Creating docker container (CPUs=2, Memory=6000MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components ...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\WINDOWS\system32> minikube addons enable metrics-server
* metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
* The 'metrics-server' addon is enabled
PS C:\WINDOWS\system32> |
```

Figura 1. Inicialização do cluster Minikube e ativação do addon *metrics-server*, habilitando a coleta de métricas de recursos nativas do Kubernetes.

Com o cluster ativo, procedeu-se com a implantação da aplicação web-app (configurada inicialmente com duas réplicas) e de seu respectivo *Horizontal Pod Autoscaler* (HPA) 2.

```
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl apply -f app-deployment.yaml
deployment.apps/web-app created
service/web-app-service created
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl apply -f hpa-config.yaml
horizontalpodautoscaler.autoscaling/web-app-hpa created
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> |
```

Figura 2. Aplicação dos manifestos de implantação e de escalonamento elástico, estabelecendo as políticas de disponibilidade da infraestrutura.

Para confirmar que a aplicação estava operando conforme o esperado e que as métricas de hardware estavam sendo lidas adequadamente pelo plano de controle, verificou-se o status dos *pods* e o consumo atual 3.

```
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-app-6b6f9d8849-29b27            1/1     Running   0           2m11s
web-app-6b6f9d8849-dfprn            1/1     Running   0           2m11s
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl top pods
NAME                                CPU(cores)   MEMORY(bytes)
web-app-6b6f9d8849-29b27            2m          8Mi
web-app-6b6f9d8849-dfprn            2m          8Mi
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> |
```

Figura 3. Verificação de estabilidade indicando os *pods* no estado de Running e a correta leitura de alocação de CPU e Memória através do comando `kubectl top pods`, validando a prontidão para os testes de estresse.

4.2. Preparação da Camada de Observabilidade (Notebook 01)

Para permitir que o Prometheus remoto coletasse o estado detalhado dos objetos do Kubernetes, instalou-se o exportador de métricas do sistema 4.

Para superar o isolamento da rede local do Minikube e permitir o tráfego das métricas para o Notebook 02, as portas de comunicação foram expostas manualmente 5 6

```

PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05\kube-state-metrics> kubectl apply -k .
serviceaccount/kube-state-metrics unchanged
clusterrole.rbac.authorization.k8s.io/kube-state-metrics unchanged
clusterrolebinding.rbac.authorization.k8s.io/kube-state-metrics unchanged
service/kube-state-metrics unchanged
deployment.apps/kube-state-metrics unchanged
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05\kube-state-metrics> kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-7d764666f9-9s56q           1/1     Running   0           6m33s
etcd-minikube                       1/1     Running   0           6m41s
kube-apiserver-minikube             1/1     Running   0           6m38s
kube-controller-manager-minikube    1/1     Running   0           6m41s
kube-proxy-l2qvt                   1/1     Running   0           6m34s
kube-scheduler-minikube             1/1     Running   0           6m41s
kube-state-metrics-654dfc96-x4mkz    1/1     Running   0           38s
metrics-server-2d74bb658-9jffh      1/1     Running   0           4m25s
storage-provisioner                 1/1     Running   0           6m36s
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05\kube-state-metrics> |

```

Figura 4. Implantação do componente kube-state-metrics no namespace kube-system e validação do seu status de execução.

```

PS C:\Users\mateu> kubectl proxy --address='0.0.0.0' --disable-filter=true
M0501 17:29:34.932649 4252 proxy.go:177] Request filter disabled, your proxy is vulnerable to XSRF attacks, please be cautious
Starting to serve on [::]:8001

```

Figura 5. Execução do comando kubectl proxy desabilitando filtros locais para abrir acesso à API.

4.3. Inicialização do Monitoramento Remoto (Notebook 02)

No segundo nó físico, o servidor Prometheus foi iniciado, conectando-se aos alvos expostos pelo Notebook 01 para consolidar a telemetria 7.

4.4. Testes de Tolerância a Falhas e Auto-Healing (Notebook 01)

Para demonstrar a resiliência da arquitetura, um dos *pods* ativos foi intencionalmente deletado para observar o comportamento do controlador 8.

O impacto dessa falha simulada e a subsequente recuperação foram registrados em tempo real pela camada de observabilidade, confirmando a eficácia da integração com o servidor de métricas 9.

4.5. Testes de Sobrecarga e Escalonamento Horizontal (HPA)

O último pilar validado foi a elasticidade. Um contêiner temporário foi utilizado para gerar um laço infinito de requisições HTTP, simulando um pico de acesso 10.

A resposta da arquitetura à sobrecarga foi acompanhada via terminal no nó de execução e pelas métricas coletadas em tempo real pelo Prometheus 11.

Para comprovar a visibilidade do evento na camada de observabilidade, acompanhou-se a evolução do consumo de processamento e a quantidade de réplicas ativas durante a execução do script de estresse 12 13.

5. Conclusão

A implantação e a execução deste projeto demonstraram com êxito a viabilidade e a robustez da orquestração de contêineres utilizando Kubernetes para a gestão de aplicações

```

PS C:\Users\mateu> kubectl port-forward -n kube-system svc/kube-state-metrics 8080:8080 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 8080

```

Figura 6. Execução do kubectl port-forward, abrindo o acesso externo para o serviço do kube-state-metrics no endereço 0.0.0.0.

```

PS C:\Users\Nutrição\Downloads\prometheus-20260501T193525Z-3-001\prometheus> .\prometheus.exe --config.file=prometheus.yml
time=2026-05-01T17:32:47.759-03:00 level=INFO source=main.go:1678 msg="updated GOGC" old=100 new=75
time=2026-05-01T17:32:47.777-03:00 level=INFO source=main.go:744 msg="Leaving GOMAXPROCS=8: CPU quota undefined" component=automa
nt=automa
time=2026-05-01T17:32:47.778-03:00 level=INFO source=memlimit.go:198 msg="GOMEMLIMIT is updated" component=automa
package=github.com/KimMachineGun/automemlimit/memlimit GOMEMLIMIT=7470349516 previous=9223372036854775807
time=2026-05-01T17:32:47.778-03:00 level=INFO source=main.go:851 msg="Starting Prometheus Server" mode=server version="(
version=3.11.3, branch=HEAD, revision=eb173f5256d4022afbale9bc3d19740a76859fae)"
time=2026-05-01T17:32:47.778-03:00 level=INFO source=main.go:856 msg="operational information" build_context="(go=go1.26
.2, platform=windows/amd64, user=root@cb3e2b0ff878, date=20260427-14:50:04, tags=builtinassets)" host_details=(windows)
fd_limits=N/A vm_limits=N/A
time=2026-05-01T17:32:47.915-03:00 level=INFO source=web.go:710 msg="Start listening for connections" component=web addr
ess=0.0.0.0:9090
time=2026-05-01T17:32:47.915-03:00 level=INFO source=main.go:1410 msg="Starting TSDB ..."
time=2026-05-01T17:32:47.923-03:00 level=INFO source=tsconfig.go:354 msg="Listening on" component=web address=[::]:909
0
time=2026-05-01T17:32:47.923-03:00 level=INFO source=tsconfig.go:357 msg="TLS is disabled." component=web http2=false
address=[::]:9090
time=2026-05-01T17:32:47.925-03:00 level=INFO source=head.go:698 msg="Replaying on-disk memory mappable chunks if any" c
omponent=tsdb
time=2026-05-01T17:32:47.925-03:00 level=INFO source=head.go:784 msg="On-disk memory mappable chunks replay completed" c
omponent=tsdb duration=0s
time=2026-05-01T17:32:47.925-03:00 level=INFO source=head.go:792 msg="Replaying WAL, this may take a while" component=ts
db
time=2026-05-01T17:32:47.925-03:00 level=INFO source=head.go:865 msg="WAL segment loaded" component=tsdb segment=0 maxSe
gment=1 duration=0s
time=2026-05-01T17:32:47.925-03:00 level=INFO source=head.go:865 msg="WAL segment loaded" component=tsdb segment=1 maxSe
gment=1 duration=0s

```

Figura 7. Execução do servidor Prometheus utilizando o arquivo de configuração `prometheus.yml`, centralizando a coleta remota sem onerar o cluster da aplicação.

```

PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl get pods
NAME READY STATUS RESTARTS AGE
web-app-6b6f9d8849-jz7kp 1/1 Running 0 57s
web-app-6b6f9d8849-kzk9z 1/1 Running 0 2m10s
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> kubectl delete pod web-app-6b6f9d8849-kzk9z
pod "web-app-6b6f9d8849-kzk9z" deleted from default namespace
PS C:\Users\mateu\Downloads\Sistemas-Distribuidos-Trabalho-05-main\Sistemas-Distribuidos-Trabalho-05-main\Trabalho-05> |

```

Figura 8. Demonstração do mecanismo de *Auto-Healing*. O log demonstra a exclusão manual de um *pod* e a imediata reação do Kubernetes (mudança de *Terminating* para a criação de um novo *pod* *Running*), garantindo a alta disponibilidade da aplicação.

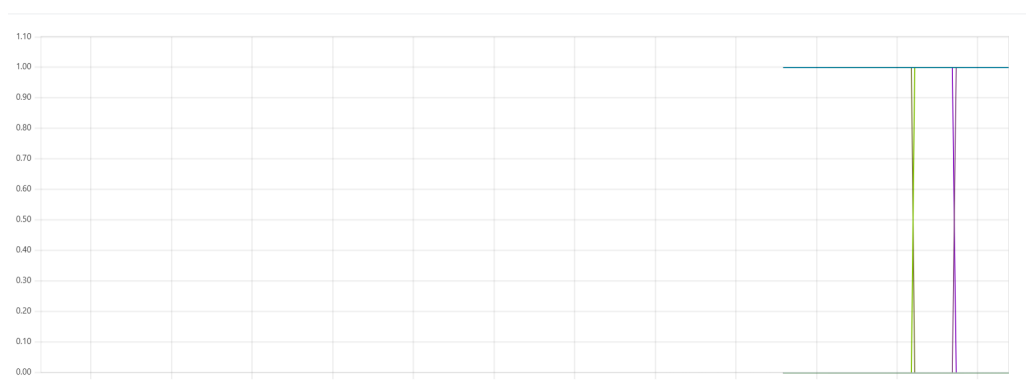


Figura 9. Monitoramento do *Auto-Healing* no Prometheus. As quedas verticais no gráfico representam o exato momento em que os *pods* foram deletados manualmente. O retorno imediato da linha ao estado anterior ilustra a rápida recriação das instâncias pelo Kubernetes, restaurando o número desejado de réplicas ativas

[illegible]

Figura 10. Injeção de carga sintética no serviço web-app-service utilizando um pod interativo (load-generator), gerando forte estresse computacional na infraestrutura.

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
web-app-hpa	Deployment/web-app	cpu: 1/50%	2	6	2	25m
web-app-hpa	Deployment/web-app	cpu: 47%/50%	2	6	2	26m
web-app-hpa	Deployment/web-app	cpu: 118%/50%	2	6	2	27m
web-app-hpa	Deployment/web-app	cpu: 118%/50%	2	6	4	28m
web-app-hpa	Deployment/web-app	cpu: 110%/50%	2	6	5	28m
web-app-hpa	Deployment/web-app	cpu: 73%/50%	2	6	5	28m
web-app-hpa	Deployment/web-app	cpu: 46%/50%	2	6	5	29m
web-app-hpa	Deployment/web-app	cpu: 1%/50%	2	6	5	30m
web-app-hpa	Deployment/web-app	cpu: 1%/50%	2	6	5	35m
web-app-hpa	Deployment/web-app	cpu: 1%/50%	2	6	2	35m

Figura 11. Terminal exibindo o monitoramento contínuo do HPA. Nota-se a detecção do aumento abrupto no uso de CPU e o consequente disparo do escalonamento dinâmico, elevando as réplicas gradativamente de 2 até o limite estipulado pela política.

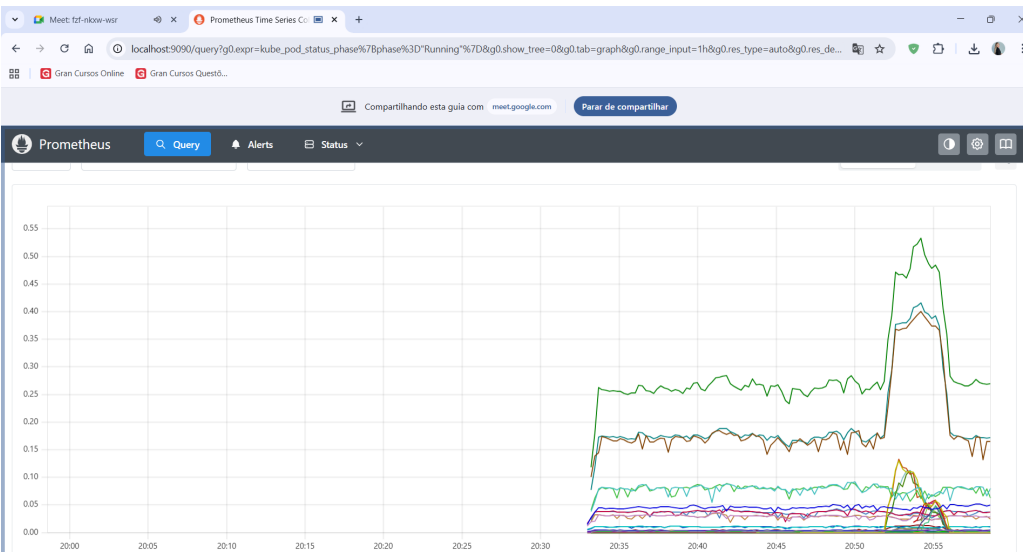


Figura 12. Painel do Prometheus evidenciando o pico agudo de consumo de CPU no cluster durante a injeção de carga. A métrica confirma que o gatilho configurado no HPA foi atingido com sucesso.



Figura 13. Gráfico da métrica `kube_hpa_status_current_replicas` no Prometheus. O salto vertical demonstra a resposta elástica do sistema provisionando novas instâncias, seguido por uma queda (scale-down) instantes após a interrupção da carga sintética, demonstrando o gerenciamento autônomo dos recursos.

distribuídas. A topologia adotada, separando o plano de execução da camada de observabilidade remota, provou ser eficaz para manter a disponibilidade e o monitoramento contínuo do sistema sob condições de estresse.

Por meio dos testes práticos, foi possível validar diretamente os pilares de resiliência e elasticidade exigidos para o ecossistema: o mecanismo de *auto-healing* garantiu a detecção e a recriação autônoma de *pods* interrompidos, enquanto o *Horizontal Pod Autoscaler* (HPA) ajustou dinamicamente a quantidade de instâncias da aplicação em resposta à injeção artificial de carga de CPU. Ademais, a integração dos componentes de métricas com o servidor Prometheus consolidou a auditoria remota do *cluster*, permitindo visualizar com clareza as oscilações de estado e consumo.

Por fim, como material complementar a este documento escrito, um vídeo de apresentação foi gravado e o respectivo *link* está sendo submetido via SIGAA. O registro audiovisual exhibe, de forma mais detalhada e dinâmica, a execução prática de todos os testes de tolerância a falhas e sobrecarga, demonstrando o comportamento do *cluster* no terminal lado a lado com a atualização simultânea das métricas e gráficos no *dashboard* do Prometheus.